SHEET 01

KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Memory & Pointers

Array bounds, owned allocations and object lifetimes, with bounded compiler and runtime captures.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2` · `-fanalyzer` · `-fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

C-MEM-01 Buffer overflow

SR-404 · Dashboard: RPM history

× BUGGY

```c
int rpm_log[8];
void log_rpm(int i, int v) {
    rpm_log[i] = v; // index8 is outside the permitted elements
}
```

▶ ASan: global-buffer-overflow

What goes wrong: This assignment requests element 8 of an eight-element array, violating its bounds. ISO C does not require a particular overwrite, crash or gauge response. Implementations and instrumentation can add checks; the language contract still requires a valid index.

> FIX

```c
#define RPM_LOG_LEN 8
int rpm_log[RPM_LOG_LEN];
bool log_rpm(size_t i, int v) {
    if (i >= RPM_LOG_LEN) return false;  // reject, don't corrupt
    rpm_log[i] = v;
    return true;
}
```

Sources: C11 draft 6.5.2.1p2;6.5.6p8;6.3.1.3; GCC14 docs Warray-bounds; GCC14 docs Wanalyzer-out-of-bounds; Clang docs Introduction;Usage;Limitations; Memory host example capture C-MEM-01 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-01 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Use an actual length and validate external signed values before converting to a size type. A size_t parameter alone is not validation. GCC14 documents selected bounds/analyzer warnings; ASan checks instrumented executed accesses. Neither a warning-free build nor a sample pass proves every index safe.

C-MEM-02 Off-by-one (fencepost)

SR-404 · Dashboard: gauge reset

× BUGGY

```c
void reset_gauges(int g[4]) {
    for (int i = 0; i <= 4; i++)   // 5 iterations
        g[i] = 0;
}
```

▶ ASan: stack-buffer-overflow

What goes wrong: With the shown zero start and unit increment, the inclusive condition requests index 4 from a four-element caller array. That access is invalid. Inclusive comparisons and length-plus-one storage are not intrinsically wrong; the actual element range and terminating-null capacity matter.

> FIX

```c
void reset_gauges(int *g, size_t n) {
    for (size_t i = 0; i < n; i++)  // half-open range [0, n)
        g[i] = 0;
}
/* call: reset_gauges(g, sizeof g / sizeof g[0]); */
```

Avoid it: A half-open loop is a useful policy when n is the actual element count. Ordinary int g[4] in this parameter adjusts to int*, carrying no array length. Caller capacity still matters. The supplied ASan capture has tool/path/symbolization scope; no exact-line diagnostic is guaranteed for every build.

Sources: C11 draft 6.7.6.3p7;6.8.5.3;6.5.6;6.7.9p14; Clang docs Usage;Limitations; Memory host example capture C-MEM-02 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-02 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-03 Use-after-free

SR-404 · Dashboard: warning text

× BUGGY

```c
char *msg = malloc(16);
if (!msg) return 1;
snprintf(msg, 16, "LOW FUEL");
free(msg);
puts(msg);            // invalid lifetime access
```

▶ gcc -Wuse-after-free=: pointer 'msg' used after 'free'

What goes wrong: free ends the allocated object lifetime and makes space available for later allocation. Old pointer values become indeterminate. Access does not have a portable stale-data or heap-corruption result, regardless of whether physical bytes appear unchanged.

> FIX

```c
char *msg = malloc(16);
if (msg == NULL) return 1;
int rc = 0;
if (snprintf(msg, 16, "LOW FUEL") < 0 || puts(msg) == EOF)
    rc = 1;
free(msg);
msg = NULL;  // clear this handle; other aliases need their own policy
return rc;
```

Sources: C11 draft 6.2.4p2;7.22.3.3;7.21.6.5;7.21.7.9; GCC14 docs Wuse-after-free;Wall; GCC14 docs Wanalyzer-use-after-free; Clang docs Introduction;Limitations; Memory host example capture C-MEM-03 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-03 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Release after intended last use and track ownership/borrowing explicitly. Clearing one handle does not repair other aliases. GCC14 -Wuse-after-free and analyzer checks request selected diagnostics; ASan does not catch every possible lifetime error.

C-MEM-04 Double free

SR-404 · ECU shutdown

× BUGGY

```c
void ecu_shutdown(char *buf) { free(buf); }
int main(void) {
    char *buf = malloc(32);
    if (buf == NULL) return 1;
    ecu_shutdown(buf);
    free(buf);  // old allocation is no longer live
}
```

▶ ASan: attempting double-free

What goes wrong: Releasing already-deallocated nonnull storage is undefined. free(NULL) is a separate, permitted no-op. A specific allocator corruption, abort message or later duplicate allocation is not an ISO C guarantee.

> FIX

```c
void ecu_shutdown(char **owner) {
    if (owner == NULL) return;
    free(*owner);
    *owner = NULL;  // clear the supplied owner handle
}
int main(void) {
    char *buf = malloc(32);
    if (buf == NULL) return 1;
    ecu_shutdown(&buf);
    ecu_shutdown(&buf);  // this call sees NULL
    free(buf);           // free(NULL) does nothing
    return 0;
}
```

Sources: C11 draft 7.22.3.3p2;6.2.4p2; Clang docs Double-free checks; GCC14 docs Wanalyzer-double-free; Memory host example capture C-MEM-04 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-04 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: A single release responsibility is an author ownership policy, not a C alias-safety type system. Document any shared/borrowed views separately. The named ASan double-free capture and GCC14 -Wanalyzer-double-free check are bounded diagnostic mechanisms.

C-MEM-05 Memory leak on an early return

SR-404 · Fuel-injection map (Ep 9)

✕ BUGGY

```c
int load_fuel_map(int ok) {
    double *map = malloc(64 * sizeof *map);
    if (!map) return -1;
    if (!ok) return -1;      // early return leaks map
    map[0] = 14.7;           /* ... use the map ... */
    free(map);
    return 0;
}
```

▸ LeakSanitizer: detected memory leaks

What goes wrong: After a successful allocation, the shown !ok return loses its only program reference without releasing the map. An early return can instead follow allocation failure, cleanup or an ownership transfer. Exhaustion timing and a particular ECU stall require the actual allocation/model/runtime contract.

> FIX

```c
int load_fuel_map(int ok) {
    int rc = -1;
    double *map = NULL;
    const size_t count = 64;
    if (count > SIZE_MAX / sizeof *map) goto out;
    map = malloc(count * sizeof *map);
    if (map == NULL || !ok) goto out;
    map[0] = 14.7;  /* exercise use */
    rc = 0;
out:
    free(map);  // includes the permitted NULL case
    return rc;
}
```

Sources: C11 draft 7.22.3;7.22.3.3;6.2.5p9;6.8.6.1; Clang docs Memory leak detection; Valgrind manual 4.2.10;4.3 leak-check; GCC14 docs Wanalyzer-malloc-leak; Memory host example capture C-MEM-05 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-05 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: A cleanup label is one way to release resources on each path; correct multiple-exit designs also exist. Clang ASan documents integrated leak checking on supported Linux configurations. Valgrind leak-check and GCC14 analyzer checks have their own options and limits; they are not all-input ownership proofs.

C-MEM-06 Returning a pointer to a local

SR-404 · Transmission: gear readout

✕ BUGGY

```c
const char *gear_name(int g) {
    char buf[8];
    snprintf(buf, sizeof buf, "G%d", g);
    return buf;            // buf's lifetime ends here
}
```

▸ gcc -Wreturn-local-addr: function returns address of local variable

What goes wrong: buf has automatic storage duration. Its lifetime ends on leaving this function, so the returned old pointer value becomes indeterminate. C does not require a stack layout, physical byte erasure or overwrite by the next function call.

> FIX

```c
int gear_name(int g, char *buf, size_t n) {
    if (buf == NULL || n == 0) return -1;
    int written = snprintf(buf, n, "G%d", g);
    return written < 0 || (size_t)written >= n ? -1 : 0;
}
/* char name[8]; if (!gear_name(3, name, sizeof name))
puts(name); */
```

Avoid it: Caller storage must remain live and have the stated capacity. This example reports formatting/truncation failure; the buffer may already be modified on failure. By-value results and static-duration constant tables are other design choices. Resolve GCC14 return-local-address warnings in their actual source context.

Sources: C11 draft 6.2.4p2-p6;7.21.6.5; GCC14 docs Wno-return-local-addr; Memory host example capture C-MEM-06 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-06 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-07 Uninitialized variable

SR-404 · Speed limiter

✕ BUGGY

```c
int clamp_speed(int raw) {
    int out;
    if (raw > 0)
        out = raw;
    return out;          // indeterminate when raw <= 0
}
```

▸ gcc -Wanalyzer-use-of-uninitialized-value: use of uninitialized value …

What goes wrong: Uninitialized automatic out has an indeterminate value. On raw<=0 this return uses the uninitialized register-eligible int and violates C11. The rule is not a description of leftover stack bytes or a promise of changing values between builds.

> FIX

```c
int clamp_speed(int raw) {
    int out = 0;         // defined default
    if (raw > 0)
        out = raw;
    return out;
}
```

Avoid it: Choose a source-level default that matches the intended logic; zero is this exercise policy. GCC14, Clang diagnostics, MemorySanitizer and Valgrind provide selected checks with analysis/instrumentation requirements. GCC14 documents that auto-var-init hardening still permits uninitialized-source warnings and optimization; it does not replace the missing logical initialization.

Sources: C11 draft 6.7.9p10;6.3.2.1p2;6.2.6.1; GCC14 docs Wuninitialized;Wmaybe-uninitialized; GCC14 docs Wanalyzer-use-of-uninitialized-value; Clang docs Wsometimes-uninitialized; Clang docs Introduction;Instrumentation;Limitations; Valgrind manual 4.2.2; GCC14 docs ftrivial-auto-var-init; Clang argument docs ftrivial-auto-var-init; Memory host example capture C-MEM-07 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-07 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-08 NULL pointer dereference

SR-404 · 4WD transfer case (Ep 8)

✕ BUGGY

```c
const struct mode *find_mode(const char *n);  // NULL if
unknown
void engage(const char *n) {
    printf("ratio %d\n", find_mode(n)->ratio);  // "4LO":
NULL!
}
```

▸ UBSan: member access within null pointer of type 'const struct mode'

What goes wrong: This fixture returns NULL for an unknown mode; other APIs have their own failure representations. Access through a null pointer is undefined, with no required physical fault. Compiler null-check optimization depends on the target and options.

> FIX

```c
const struct mode *find_mode(const char *n); // fixture
contract
int engage(const char *n) {
    if (n == NULL) return -1;
    const struct mode *m = find_mode(n);
    if (m == NULL) return -1;
    return printf("ratio %d\n", m->ratio) < 0 ? -1 : 0;
}
```

Sources: C11 draft 6.5.3.2p4;6.5.2.3;7.21.6.1p14; GCC14 docs fdelete-null-pointer-checks; GCC14 docs Wnull-dereference; GCC14 docs Wanalyzer-null-dereference; Clang docs null; Memory host example capture C-MEM-08 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-08 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Read each API contract, including borrowed-object lifetime and failure status. This fixture only checks a lookup and prints a ratio; it does not prove a vehicle-mode transition. GCC14 analyzer/null warnings and UBSan document selected checks, not complete validation of every pointer or path.

C-MEM-09 realloc() result overwrites the only pointer

SR-404 · Trip computer log

× BUGGY
```c
static char *trip_log; // driver initially allocates64 bytes
int grow_log(size_t want) {
    trip_log = realloc(trip_log, want); // sole owner
overwritten
    return trip_log ? 0 : -1;
}
```

▸ LeakSanitizer: detected memory leaks

What goes wrong: For an existing allocation and a nonzero request, allocation failure preserves the old object. Overwriting the sole owner with NULL then loses it. C11 zero-size allocation rules are separate; N3096 C23 makes zero-size realloc undefined in its nonnull-pointer branch. This helper rejects every zero request by policy.

> FIX
```c
static char *trip_log;  // owned handle; initially NULL
int grow_log(size_t want) {
    if (want == 0) return -1;
    char *tmp = realloc(trip_log, want);
    if (tmp == NULL) return -1; // owner is unchanged
    trip_log = tmp;
    return 0;
}
```

Avoid it: Retain the owner until positive-size allocation succeeds. Validate count/element-size multiplication before requesting bytes; POSIX2024 specifies reallocarray for that task. allocator_may_return_null permits a supported sanitizer allocator to return failure instead of terminating; it does not force every request to fail.

Sources: C11 draft 7.22.3p1;7.22.3.5p2-p4;6.2.5p9; C23 draft N3096 7.24.3.7p3 nonnull-pointer zero-size branch; POSIX2024 DESCRIPTION;ERRORS; Sanitizer runtime docs allocator_may_return_null;detect_leaks; Memory host example capture C-MEM-09 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-09 defined input/cleanup groups or asserted-host scope; transformations recorded

C-MEM-10 sizeof on a pointer, not the array

SR-404 · Sensor sample buffer

× BUGGY
```c
void zero_samples(int samples[16]) {
    memset(samples, 0, sizeof samples); // parameter is a
pointer
}
```

▸ gcc -Wsizeof-array-argument: 'sizeof' on array function parameter 'sam…

What goes wrong: The ordinary array parameter is adjusted to int*, so sizeof samples reports a pointer size. The exact cleared-byte count is a target property. A driver that leaves most samples initially zero cannot by itself demonstrate that the whole array was cleared.

> FIX
```c
bool zero_samples(int *samples, size_t n) {
    if (n > SIZE_MAX / sizeof *samples) return false;
    if (n != 0 && samples == NULL) return false;
    for (size_t i = 0; i < n; ++i) samples[i] = 0;
    return true;
}
#define ARRAY_LEN(a) (sizeof (a) / sizeof (a)[0]) // actual
arrays only
/* int s[16]; zero_samples(s, ARRAY_LEN(s)); */
```

Sources: C11 draft 6.7.6.3p7;6.5.3.4;6.2.5p9;6.2.6.2p5;7.22.3.2 footnote296;7.24.6.1; GCC14 docs Wsizeof-array-argument;Wsizeof-pointer-memaccess; Memory host example capture C-MEM-10 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-10 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: The caller still supplies storage for n int elements; these checks cannot discover an arbitrary allocation capacity. The typed loop assigns integer zero and permits an empty NULL range. ARRAY_LEN is not a pointer-rejecting type check. Byte-count products can wrap, and a zero byte pattern is not a semantic zero for every replacement type. GCC14 documents selected sizeof diagnostics.

C-MEM-11 Stale pointer after realloc moved the block

SR-404 · Fault-code list

× BUGGY
```c
int *v = malloc(4 * sizeof *v);
if (!v) return 1;
int *first = &v[0];              // alias into the block
int *tmp = realloc(v, 4096 * sizeof *v);
if (!tmp) { free(v); return 1; }
v = tmp;  // successful resize replaces the old object
*first = 1;  // old alias is invalid after success
free(v);
```

▸ gcc -Wuse-after-free=: pointer 'first' may be used after 'realloc'

What goes wrong: Successful realloc replaces the old object, even when the returned numeric address is the same. Old aliases cannot be evaluated as continuing pointers to it. Positive-size allocation failure is different: it leaves the old object allocated.

> FIX
```c
int *v = NULL;
size_t count = 4, grown = 4096, first = 0;
if (count > SIZE_MAX / sizeof *v || grown == 0 ||
    grown > SIZE_MAX / sizeof *v) return 1;
v = malloc(count * sizeof *v);
if (v == NULL) return 1;
int *tmp = realloc(v, grown * sizeof *v);
if (tmp == NULL) { free(v); return 1; }
v = tmp;
if (first >= grown) { free(v); return 1; }
v[first] = 1; // derive from the returned live base
free(v);
```

Sources: C11 draft 6.2.4p2;7.22.3.5p2-p4;6.5.6; Clang docs Introduction;Limitations; C++17 draft N4659 26.3.11.3 [vector.capacity]p6,p10; Memory host example capture C-MEM-11 exact supplied-example diagnostic/result; Defined memory host checks C-MEM-11 defined input/cleanup groups or asserted-host scope; transformations recorded

Avoid it: Keep an index/offset when suitable, validate it against the new extent, then derive a pointer from the returned base. Do not compare an old alias after success to test whether it moved. ASan captures selected invalid accesses, not complete lifetime provenance. C++17 vector reallocation has a specific invalidation contract; other container operations need their own rules.

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.